

Approach Documentation: 10M Row Topic Classification

Data Processing

Handling a 4GB Parquet file containing 10 million rows immediately presented a massive memory constraint. Attempting to load this dataset into standard Pandas structures resulted in fatal Out-Of-Memory (OOM) errors.

- **Data Loading Strategy:** I implemented a memory-efficient streaming pipeline utilizing `pyarrow.parquet.iter_batches`. This allowed the dataset to be ingested in chunks of 50,000 rows, strictly capping RAM usage.
- **Preprocessing & Tokenization:** Text normalization (lowercasing, regex-based alphanumeric filtering) was applied on the fly.
- **Vocabulary Building:** Initially, iterating through all 10 million rows just to build the vocabulary caused a massive CPU bottleneck. I implemented an early-stopping sampling strategy, scanning only the first 100,000 rows to generate a robust vocabulary of 100,000 unique words, mapping rare words to `<UNK>` and sequence buffers to `<PAD>`.

Exploration & Iteration

My experimentation journey prioritized balancing the strict parameter constraints with memory-safe execution.

- **Attempt 1: HashingVectorizer + SGDClassifier:** My first instinct for a dataset this large was an out-of-core classical ML approach. Using `partial_fit` was incredibly memory efficient. However, it completely failed to capture deeper semantic relationships in the text, leading to a poor F1 score.
- **Attempt 2: The FastText-style PyTorch Approach (Final Selection):** I pivoted to a custom deep learning architecture built from scratch. Standard recurrent models (LSTMs) were computationally unfeasible for 10 million rows. I opted for PyTorch's `nn.EmbeddingBag`. By computing the mean of the embeddings directly, I completely bypassed the need to pad tensors to maximum sequence lengths, drastically cutting down GPU matrix operation overhead and memory footprint.

Architecture Details (Final Model)

The final architecture is a lightweight, custom feed-forward network utilizing embedding bags.

- **Model Structure:**

- **Layer 1:** `nn.EmbeddingBag` (Vocab size: 100,000 | Dimension: 64)
- **Layer 2:** `nn.Linear` (Transforms to 128 hidden units)
- **Activation & Regularization:** `ReLU + Dropout(0.3)`
- **Output Layer:** `nn.Linear` (128 units mapped to target classes)

- **Parameter Count:** The assignment required staying under 5 Billion parameters. Assuming $V = 100,000$, $D = 64$, and $C = 10$ classes, the calculation is:

$$Parameters = (V \times D) + (D \times 128 + 128) + (128 \times C + C)$$

$$Parameters = (100,000 \times 64) + (64 \times 128 + 128) + (128 \times 10 + 10)$$

$$Parameters = 6,400,000 + 8,320 + 1,290 = 6,409,610$$

The model holds exactly 6.4 million parameters, which is barely 0.12% of the maximum allowed limit. The compiled weights file is highly efficient, coming in at just 28 MB.

Training Strategy

- **Train/Validation Split:** Due to the single-pass streaming architecture, a temporal split was utilized to evaluate the model.
- **Hyperparameters:** I used the Adam optimizer (Learning Rate: 0.005) for fast convergence. The batch size was set to 10,000 to maximize T4 GPU utilization.
- **Hardware and Time:** Training was executed on a Google Colab T4 GPU. By fixing the vocabulary bottleneck, the entire 10 million row dataset finished a full training epoch in a highly efficient timeframe.

Evaluation Metrics

After processing the streamed data for just a single epoch, the model achieved highly robust baseline metrics:

- **Accuracy:** 0.8136
- **Precision:** 0.8108
- **Recall:** 0.8136
- **F1 Score:** 0.8082

Error Analysis

Rigorous inference testing revealed the model's distinct strengths and areas for potential refinement:

- **High Confidence Predictions:** On distinct topics, the model exhibits near-perfect confidence. For example, testing political text yielded `politics` at 100.00% confidence, and culinary text yielded `food_and_dining` at 99.99%.
- **Dataset Anomalies:** The model successfully learned and reproduced typographical errors natively present in the dataset's target labels (e.g., classifying hardware strings as `electronics_and_hardare`).
- **Misclassification Patterns & Improvements:** Sequences dominated by highly specialized jargon sometimes triggered the `<UNK>` token mapping, causing the model to default toward majority classes. A future iteration could implement subword tokenization (like BPE) from scratch to handle out-of-vocabulary terms more gracefully without inflating the parameter count.